

Animation

DongJoon Kim

| Phone : +82-2-940-8461

| Email : dongjoonkim@kw.ac.kr



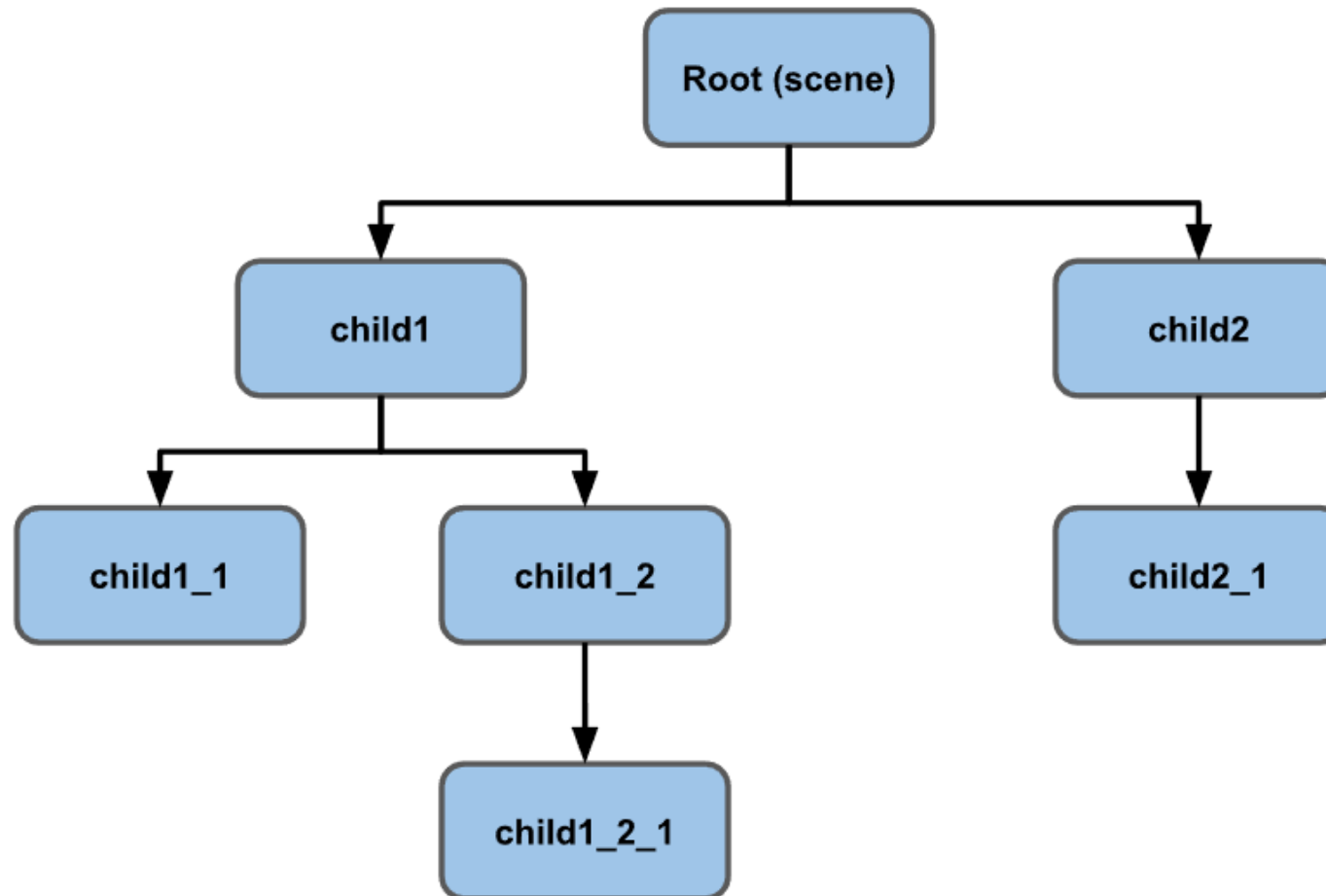
Motion Capture to Animation



Revisit:
Scene Graph and Local Transform

Scene Graph (Tree)

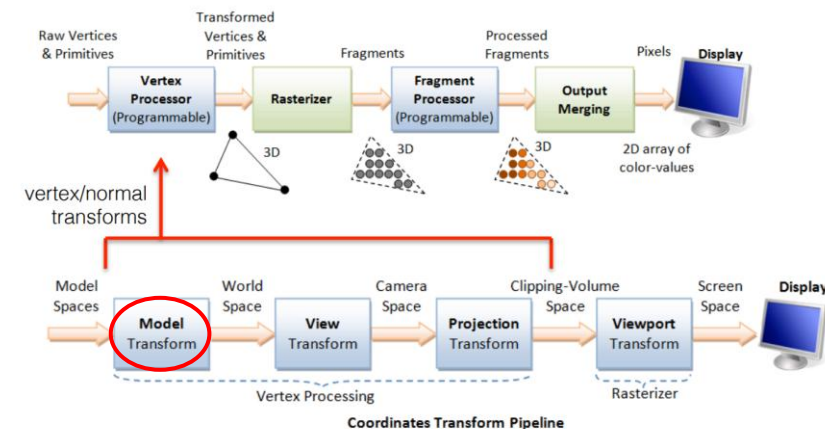
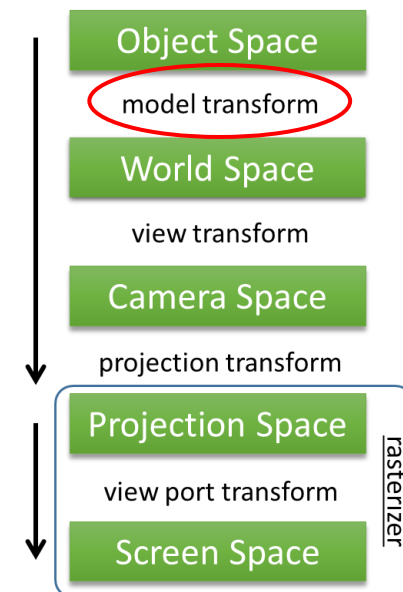
- Model transforms are highly related to the 3D scene tree structure (i.e., Scene Graph)
 - A tree structure contains '**hierarchical**' nodes (parent and children)



Scene graph

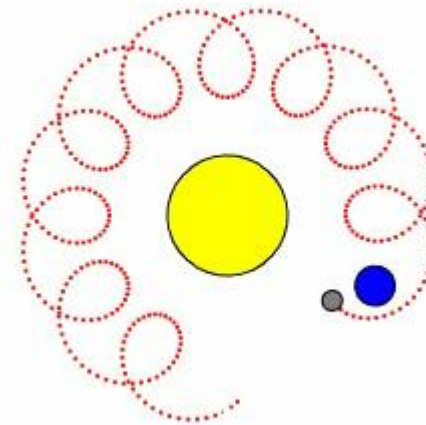
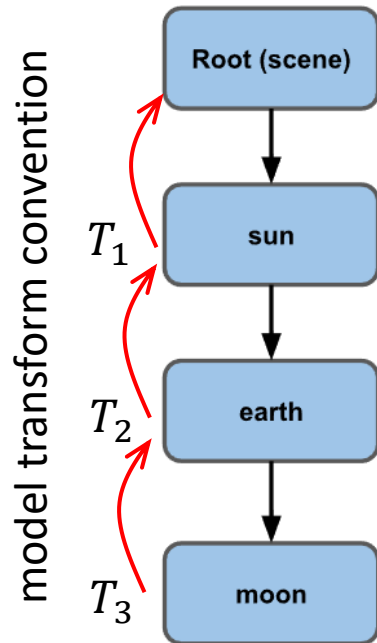
Revisit: Model Transformations

- Most transformation is handled as a model transformation
- Scaling transform
 - When the unit of OS and WS is different
 - When the scene requires the resized model
- Translation
 - Just move the model in WS
- **Rotation** (we will discuss this in more detail later 😊)
 - Orientation setting
- The combination of these transforms is defined as affine transforms!



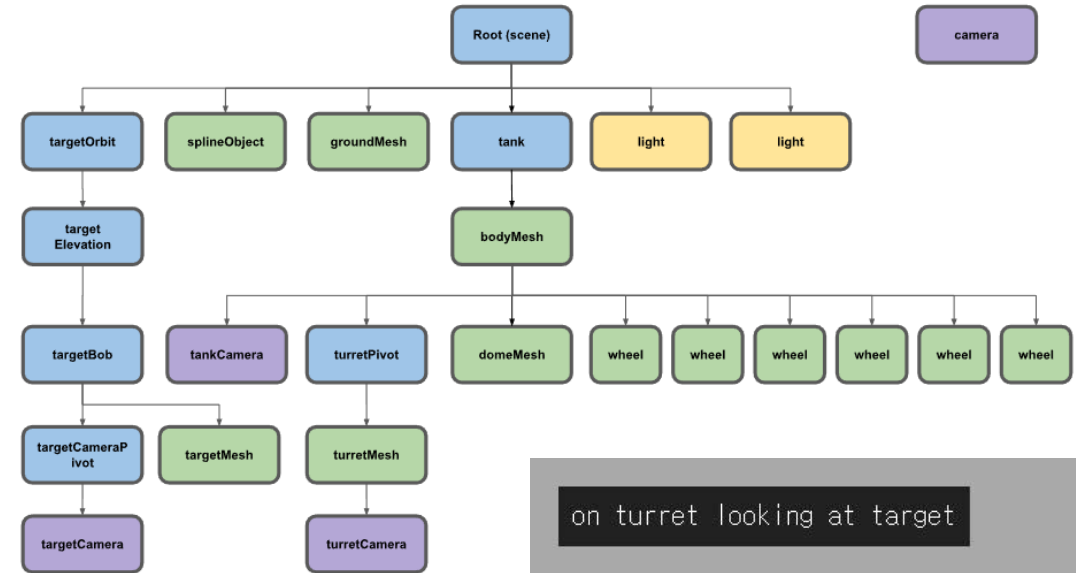
Revisit: Local Transform vs. World Transform

- Transform to the root (scene or world) is called **World Transform**
- Transform to the parent (instead of the world) is called **Local Transform**
- Local transform conveniently describes the hierarchical item's local location and orientation
 - Scene items (e.g., cameras, actors, lights, ...), Skeleton bones
 - Bottom-top transform for the world transform



T_2 is described as the earth's local transform to the sun

Revisit: Scene Graph (Tree) Examples



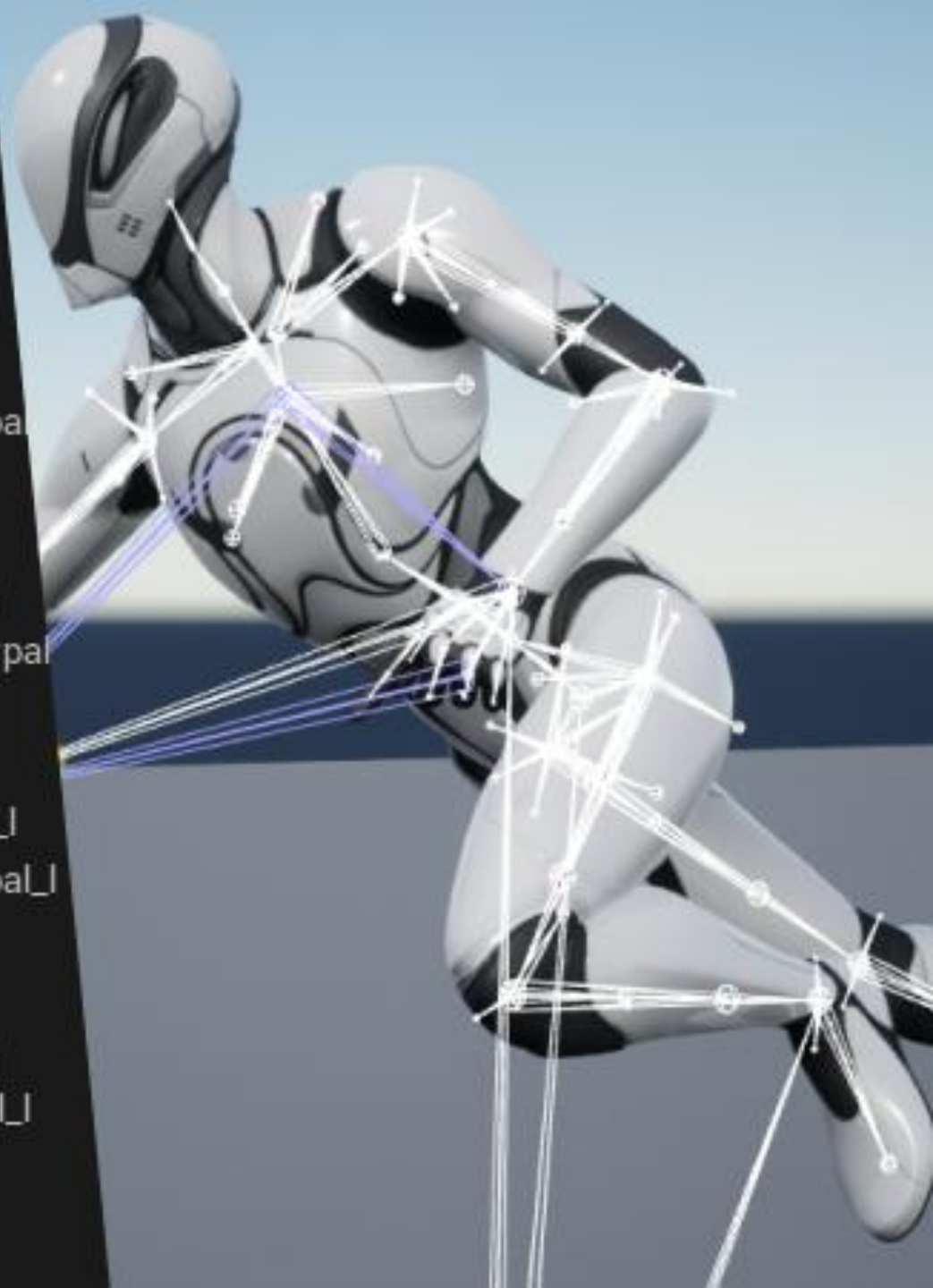
on turret looking at target



Skeleton for Animation

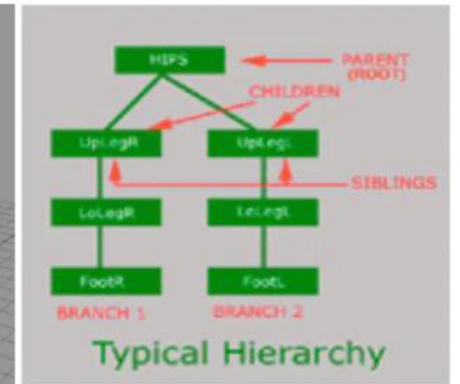
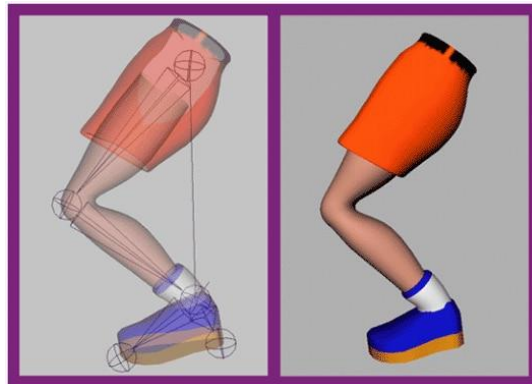
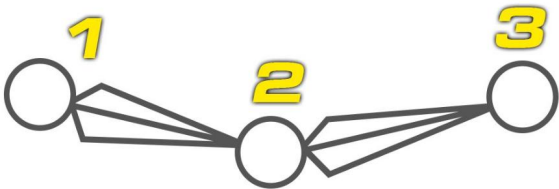


- spine_05
- clavicle_l
 - clavicle_out_l
 - clavicle_scap_l
- upperarm_l
- lowerarm_l
- hand_l
 - index_metacarpal
 - index_01_l
 - index_02_l
 - index_03_l
 - middle_metacarpal
 - middle_01_l
 - middle_02_l
 - middle_03_l
 - pinky_metacarpal_l
 - pinky_01_l
 - pinky_02_l
 - pinky_03_l
 - ring_metacarpal_l
 - ring_01_l
 - ring_02_l
 - ring_03_l



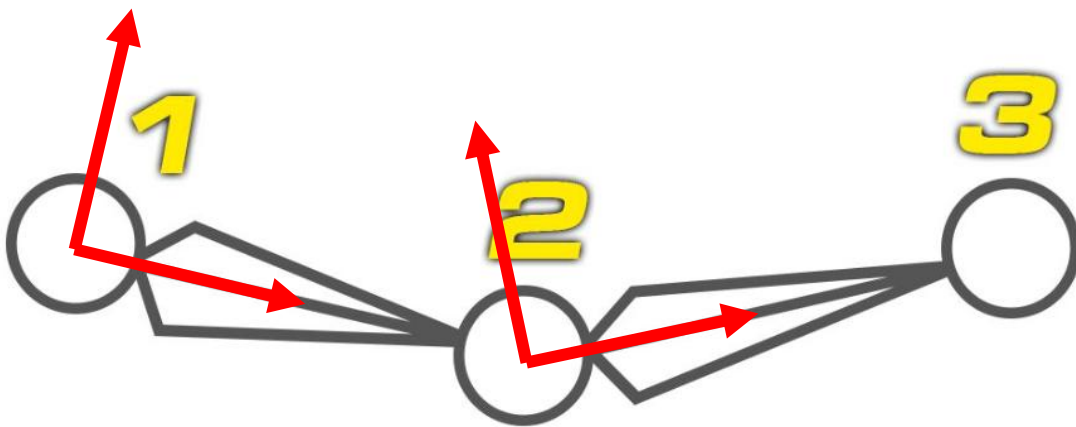
Skeleton

- A **joint hierarchy** for character posing and animations using deformations
 - A fundamental structure for CG (Computer-Generated) animation models
- A **joint** can be connected to multiple joints (children)
 - The connected joints are the children of the joint
 - A joint is also referred as a bone
 - Defined by a head point and a tail point (i.e., single line structure)
- A joint has a **local transform** (to its parent frame)
 - Composed of a rotation and a translation

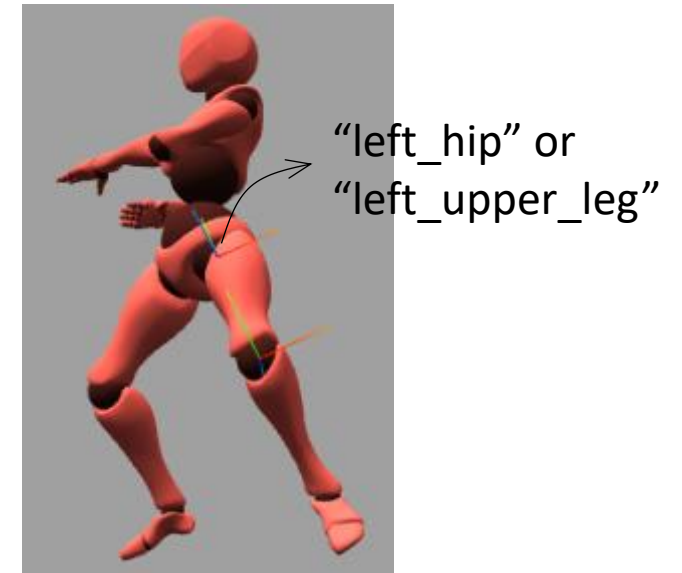


Joint vs. Bone

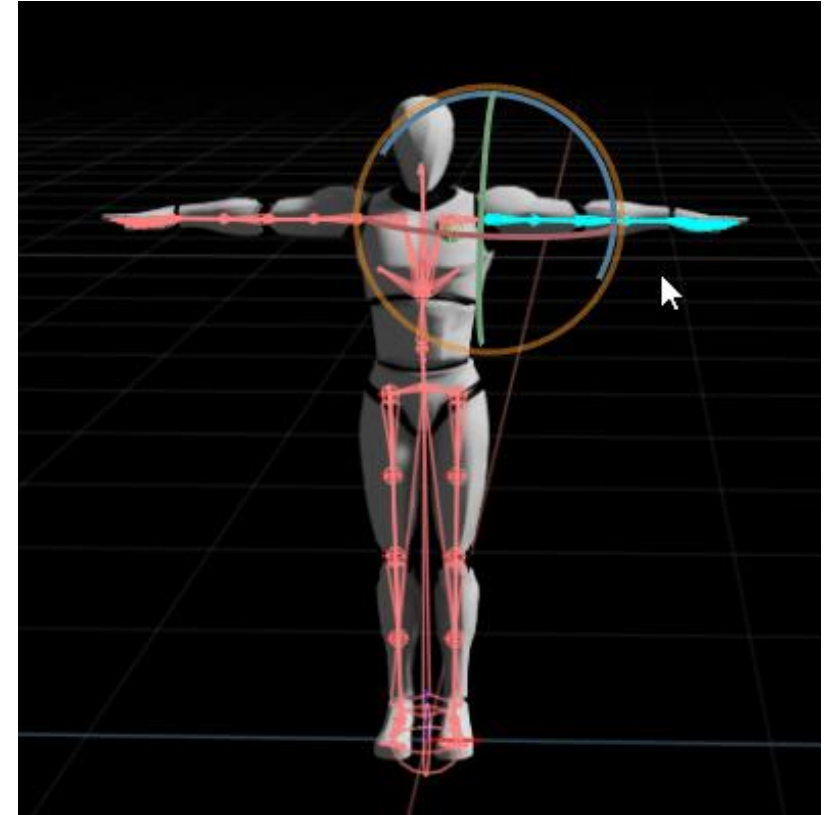
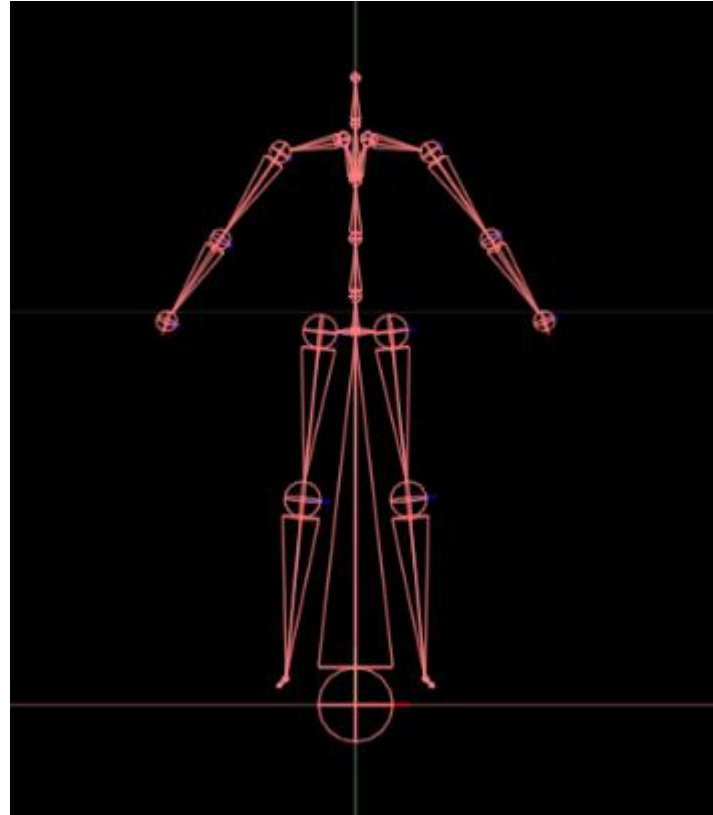
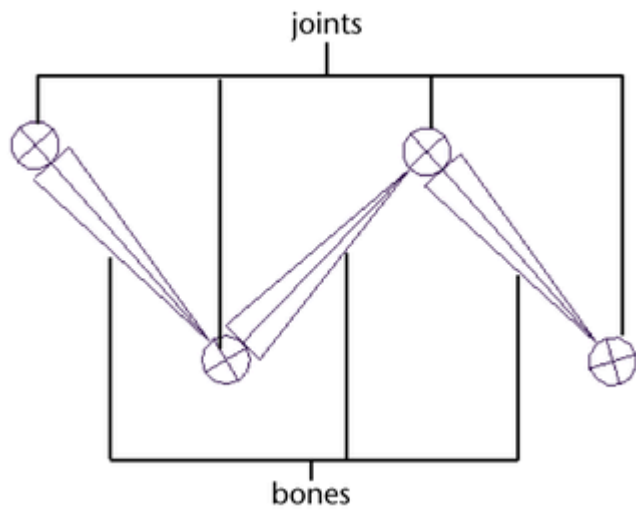
- Joint is more straightforward to understand the skeletons
 - Note: a joint can be connected to multiple joints
- In CG skeleton, a bone is a set of lines starting from its joint to its children (joints)
- Naming: normally use the joint itself or its bone
- A joint has a static offset with respect to its parent frame under the startup (e.g., T) pose
 - Note: its local position is determined by the offset w.r.t. its direct parent frame
 - i.e., the local transform of a joint determines the positions of its children joints



Joint2 has its local transform to the Joint1's frame (reference frame)
Joint2's bone refers to the line from Joint2 to Joint3



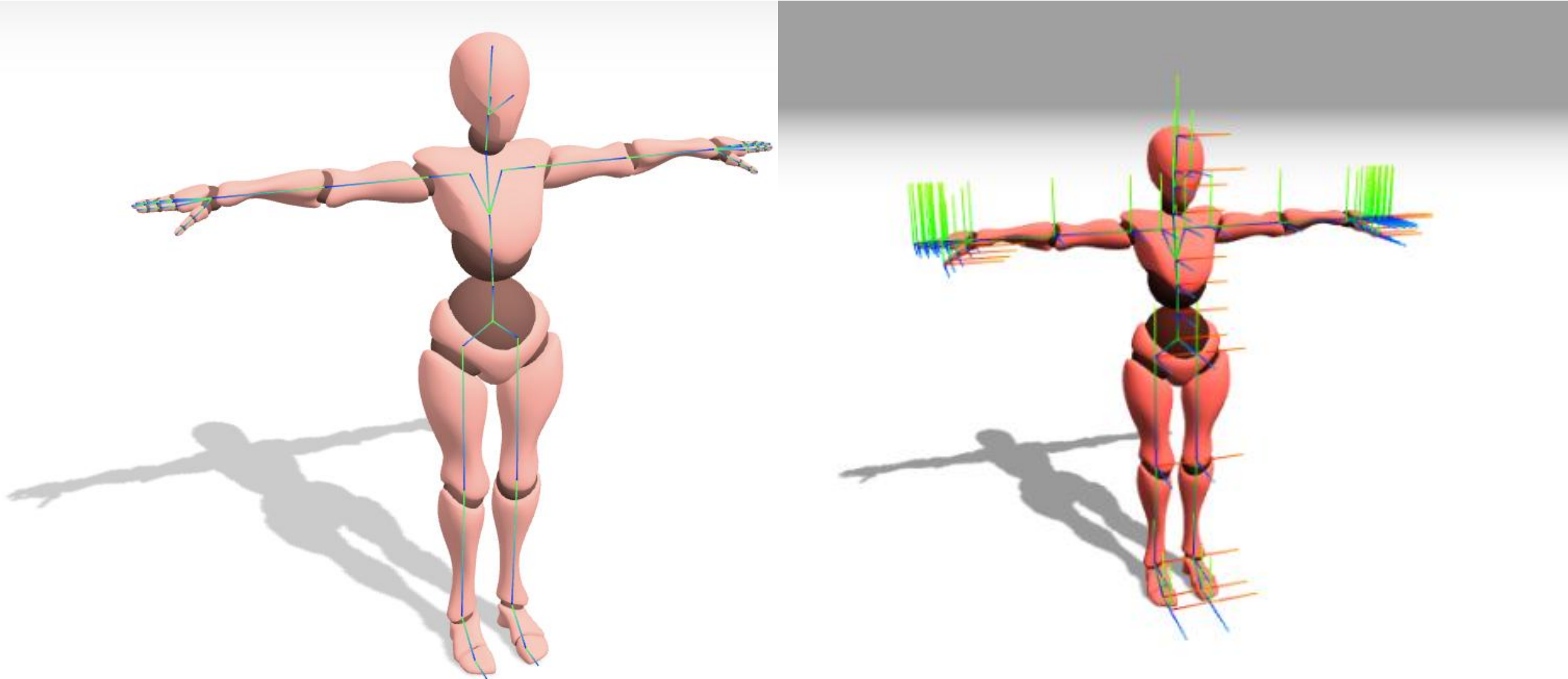
Joint vs. Bone



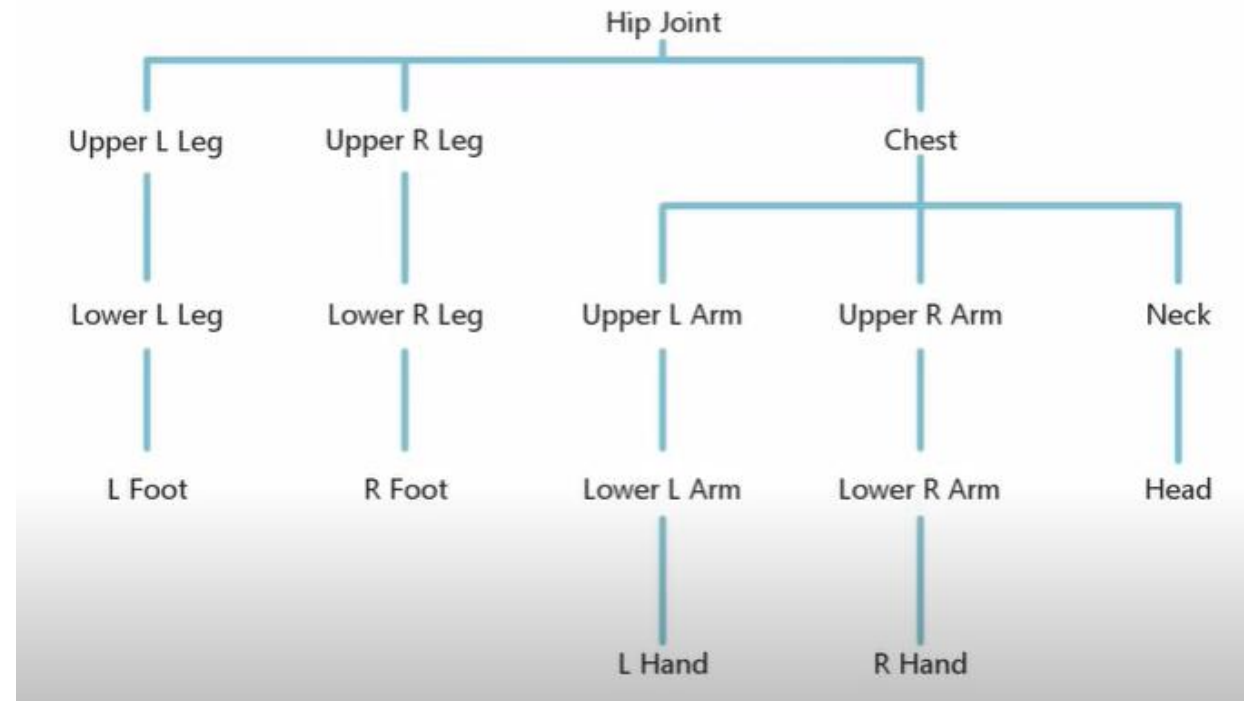
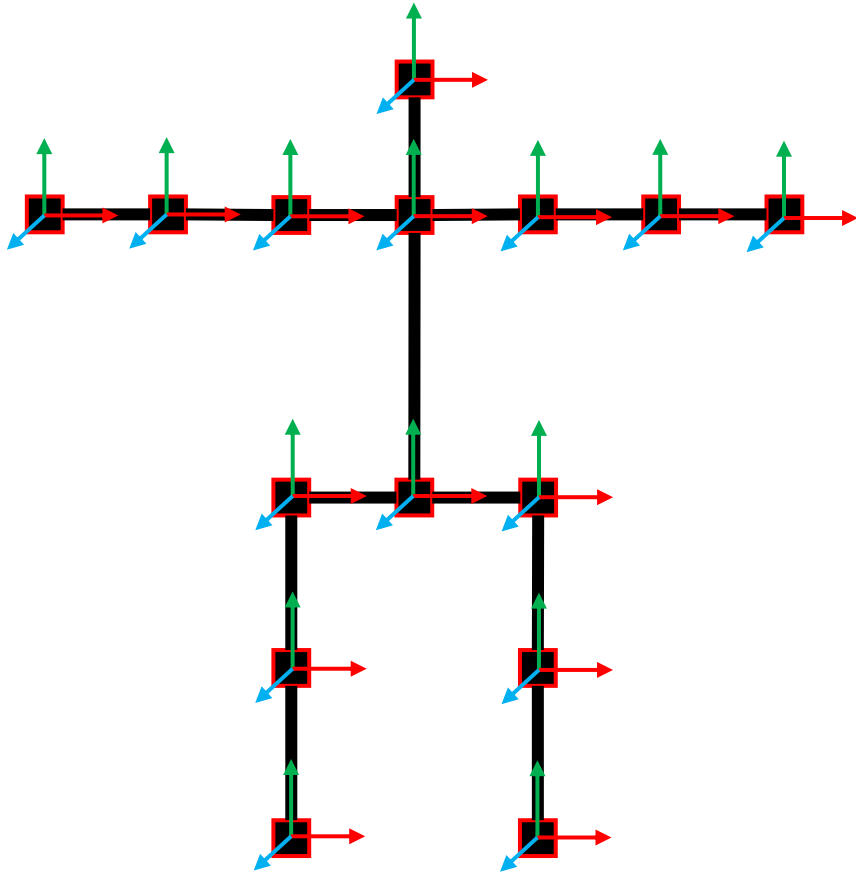
CG Skeletons and Bones

- CG Skeleton is defined by a collection of bones or a joint hierarchy
- In some tools (e.g., Blender), **Skeleton** is also called “**Armature**”
- Worth reading:
 - <https://www.intel.com/content/www/us/en/developer/articles/technical/character-animation-skeletons-and-inverse-kinematics.html>
- Web example
 - https://threejs.org/examples/#webgl_animation_skinning_blending

Hierarchical Joints (Bones) for Skeleton



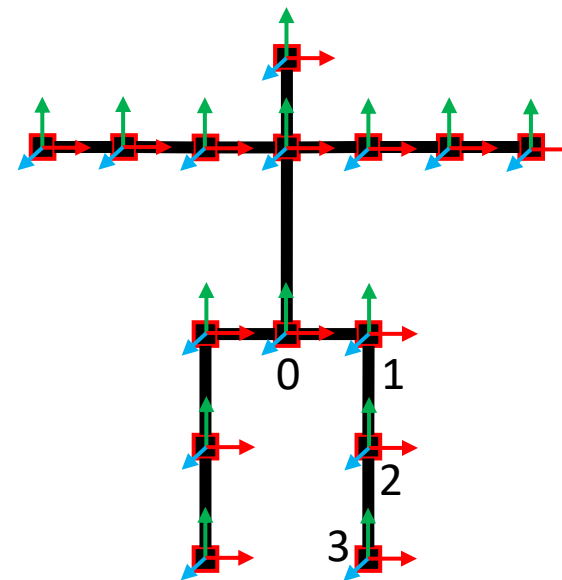
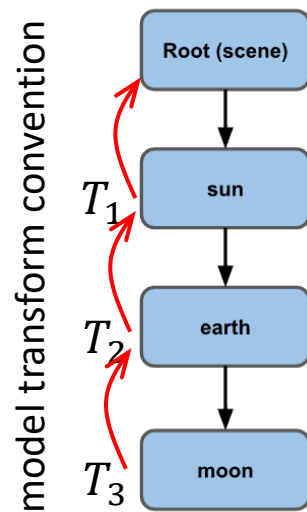
Initial Pose and Place Axes at each Joint



In this setup pose (typically using T pose), the offset of each joint w.r.t its direct parent frame is determined

Hierarchical Local Transforms

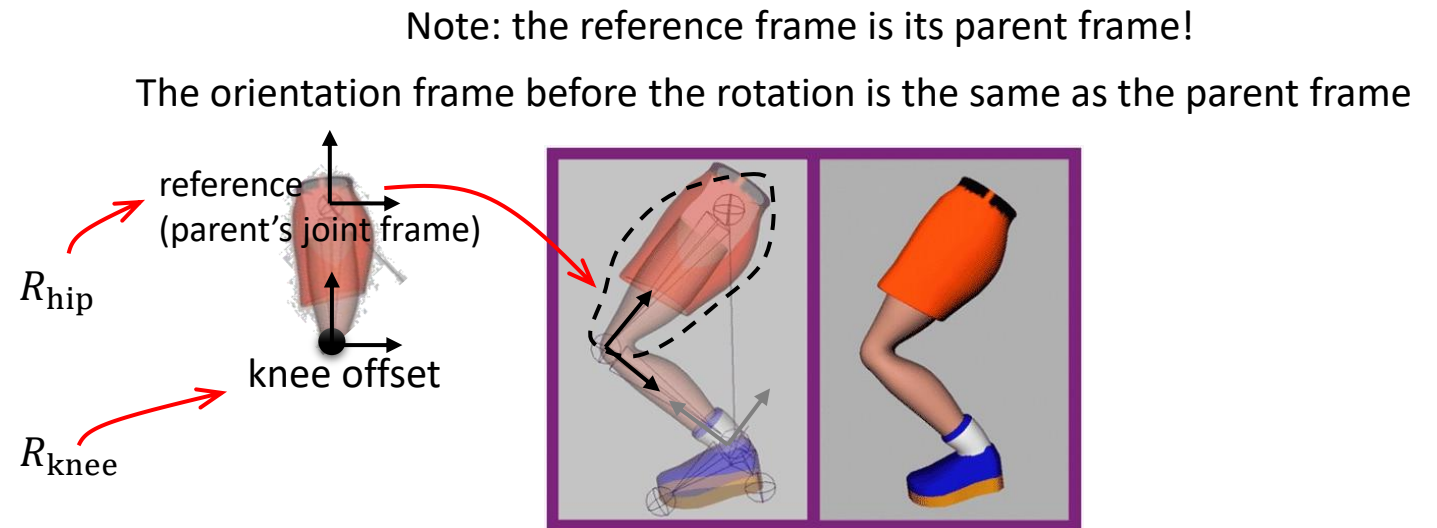
- In a scene graph hierarchy, the world transform can be calculated throughout the bottom-top local transforms
 - E.g., $\mathbf{p}_3 = \mathbf{p}_0 + \mathbf{R}_0 \mathbf{j}_1 + \mathbf{R}_0 \mathbf{R}_1 \mathbf{j}_2 + \mathbf{R}_0 \mathbf{R}_1 \mathbf{R}_2 \mathbf{j}_3$ (refer to the figure below)
 - $\mathbf{j}_i$ refers to the offset of joint i w.r.t. its direct parent!
 - $\mathbf{R}_i$ refers to the rotation about joint i
 - Then, the local transform of joint i is $\mathbf{R}_{i-1} \mathbf{T}(\mathbf{j}_i)$ (which is in col-major convention)
 - The calculation can be simply done by creating a chain of matrix multiplication



the most important key point!!
to understand the rigging transforms

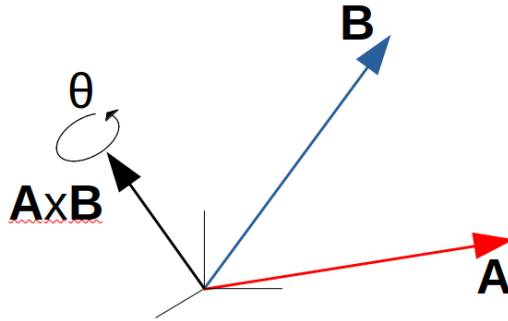
Estimating Joint (Bone) Angles

- Note that each joint (bone) has a specific (static) offset (no change for rigid skeletons)
 - The offset defines a translation
 - A local transform of a joint is defined by
 - First, applying a rotation of its parent joint (describing the orientation of the joint)
 - Second, applying a translation defined by the joint's offset
 - Here, the translation is given, and the rotation must be calculated



Estimating Joint (Bone) Angles

- The joint rotation can be calculated by
 - Rotation between two vectors or Tensor product



Use the following geometric operators...

1. Calculate the rotation axis by cross-product A and B
2. Calculate the in-between angle of A and B
3. Make a matrix from the above rotation parameters

But the individual operator implies some redundant calculations

Make the solution more efficiency using more fundamental mathematics! 😊

```
#calculate rotation matrix to take A vector to B vector
def Get_R(A,B):

    #get unit vectors
    uA = A/np.sqrt(np.sum(np.square(A)))
    uB = B/np.sqrt(np.sum(np.square(B)))

    #get products
    cos_t = np.sum(uA * uB)
    sin_t = np.sqrt(np.sum(np.square(np.cross(uA,uB)))) #magnitude

    #get new unit vectors
    u = uA
    v = uB - np.sum(uA * uB)*uA
    v = v/np.sqrt(np.sum(np.square(v)))
    w = np.cross(uA, uB)
    w = w/np.sqrt(np.sum(np.square(w)))

    #get change of basis matrix
    C = np.array([u, v, w])

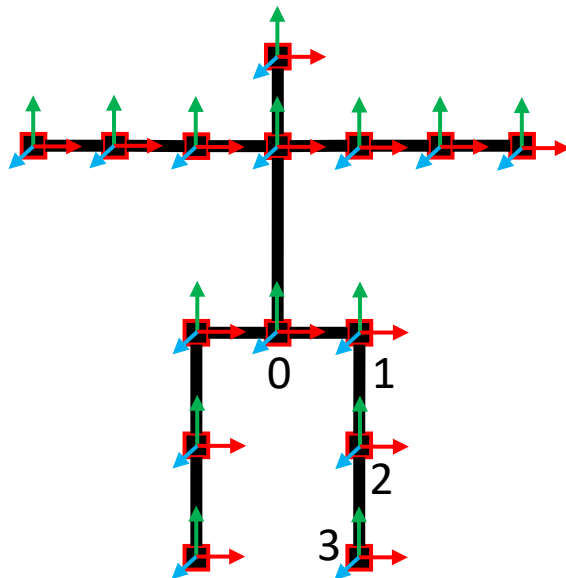
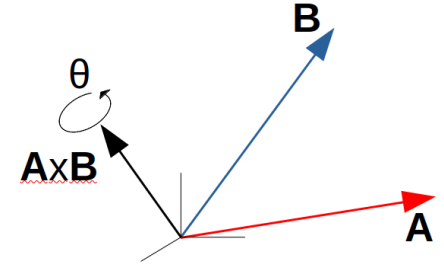
    #get rotation matrix in new basis
    R_uvw = np.array([[cos_t, -sin_t, 0],
                      [sin_t, cos_t, 0],
                      [0, 0, 1]])

    #full rotation matrix
    R = C.T @ R_uvw @ C
    #print(R)
    return R
```

THREE.js math provides this! But you can implement by yourself!

Estimating Joint (Bone) Angles

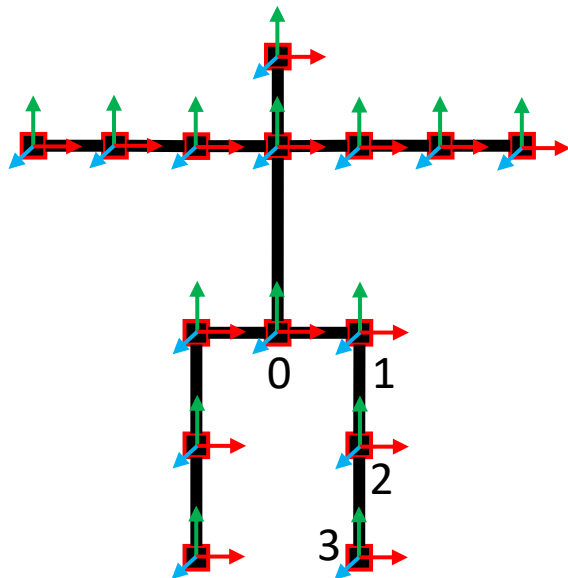
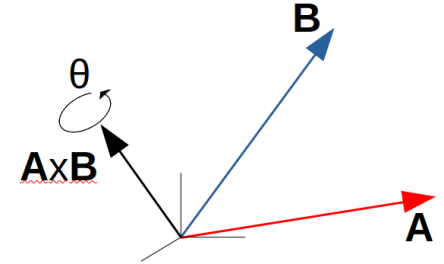
- The joint rotation can be calculated by
 - Rotation between two vectors or Tensor product
 - This is based on FK (note each $\mathbf{j}_i$ is determined in the T pose)
 - First, determine $\mathbf{p}_0$ (skeleton's root position) directly from the landmark
 - Second, compute $\mathbf{R}_0$ from the equation for $\mathbf{p}_1$ (using the rotation btw. two vectors)



$$\begin{aligned}\mathbf{p}_1 &= \mathbf{p}_0 + \mathbf{R}_0 \mathbf{j}_1 \\ \Rightarrow \mathbf{p}_1 - \mathbf{p}_0 &= \mathbf{R}_0 \mathbf{j}_1 \\ \Rightarrow \mathbf{v}_{01} &= \mathbf{R}_0 \mathbf{j}_1 \quad (\text{note: } \|\mathbf{v}_{01}\| = \|\mathbf{j}_1\|)\end{aligned}$$

Estimating Joint (Bone) Angles

- The joint rotation can be calculated by
 - Rotation between two vectors or Tensor product
 - This is based on FK (note each $\mathbf{j}_i$ is determined in the T pose)
 - First, determine $\mathbf{p}_0$ (skeleton's root position) directly from the landmark
 - Second, compute $\mathbf{R}_0$ from the equation for $\mathbf{p}_1$ (using the rotation btw. two vectors)
 - Calculate other joint angles of further (child) joints



$$\mathbf{p}_1 = \mathbf{p}_0 + \mathbf{R}_0 \mathbf{j}_1$$

$$\rightarrow \mathbf{v}_{01} = \mathbf{R}_0 \mathbf{j}_1 \quad (\text{note: } \|\mathbf{v}_{01}\| = \|\mathbf{j}_1\|)$$

$$\mathbf{p}_2 = \mathbf{p}_0 + \mathbf{R}_0 \mathbf{j}_1 + \mathbf{R}_0 \mathbf{R}_1 \mathbf{j}_2 = \mathbf{p}_1 + \mathbf{R}_0 \mathbf{R}_1 \mathbf{j}_2$$

$$\rightarrow \mathbf{p}_2 - \mathbf{p}_1 = \mathbf{R}_0 \mathbf{R}_1 \mathbf{j}_2$$

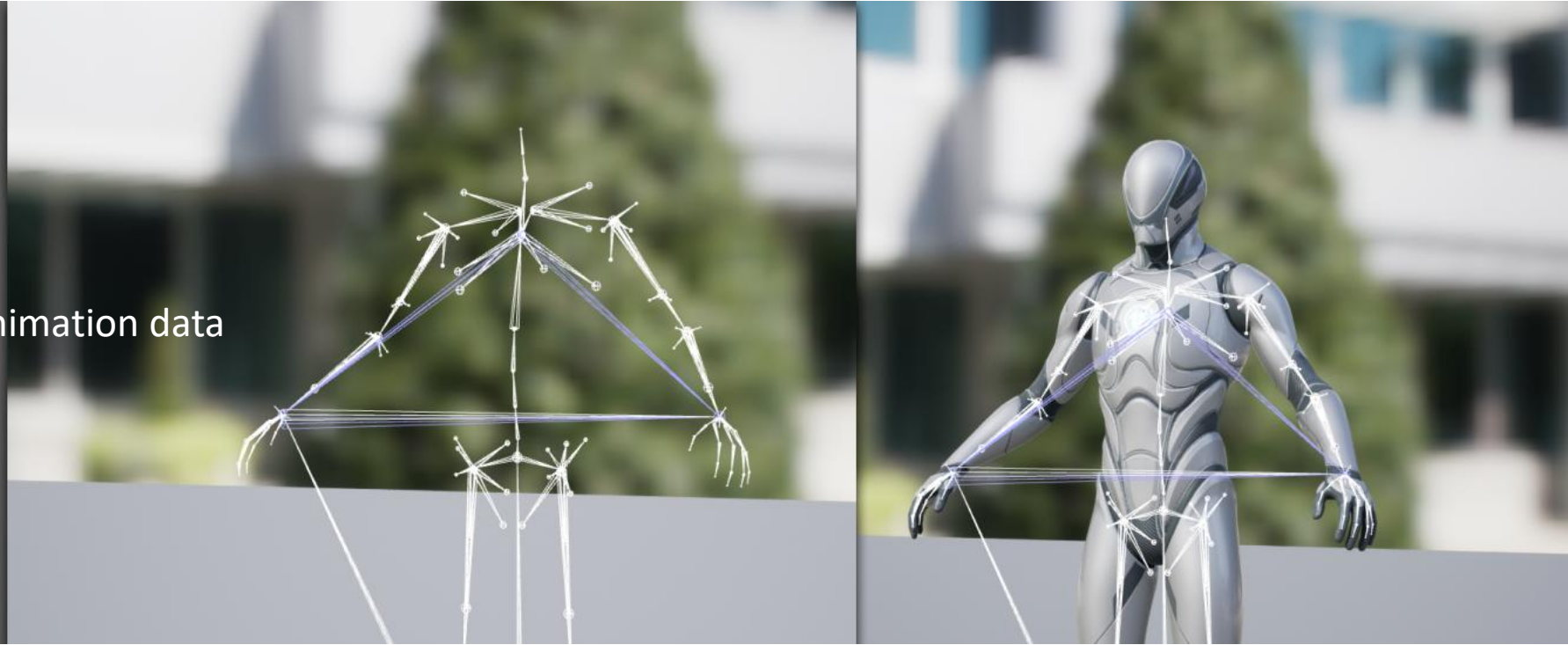
$$\rightarrow \mathbf{R}_0^{-1} \mathbf{v}_{12} = \mathbf{R}_1 \mathbf{j}_2 \quad (\text{note: } \mathbf{R}^{-1} = \mathbf{R}^T)$$

$$\rightarrow \mathbf{v}'_{12} = \mathbf{R}_1 \mathbf{j}_2 \quad (\text{note: } \|\mathbf{v}'_{12}\| = \|\mathbf{j}_2\|)$$

Unreal Engine Skeletons

Skeletons

Learn about Skeletons, Bones, and animation data management in Unreal Engine.

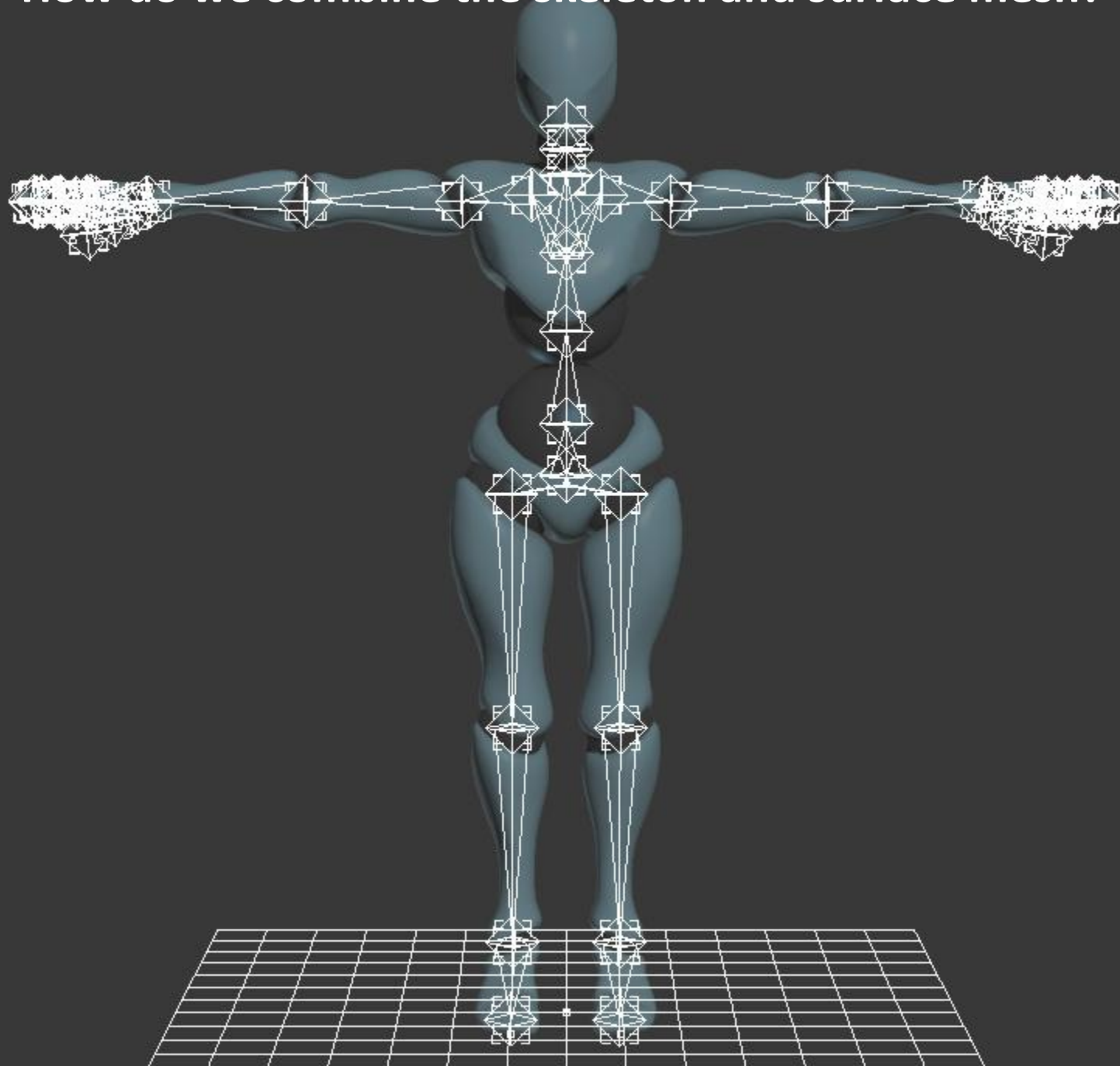


<https://docs.unrealengine.com/5.0/en-US/skeletons-in-unreal-engine/>

Simple trial : <https://youtu.be/FebmlYAJ0Uc?si=hUV8qU6i8DMgtt9a>

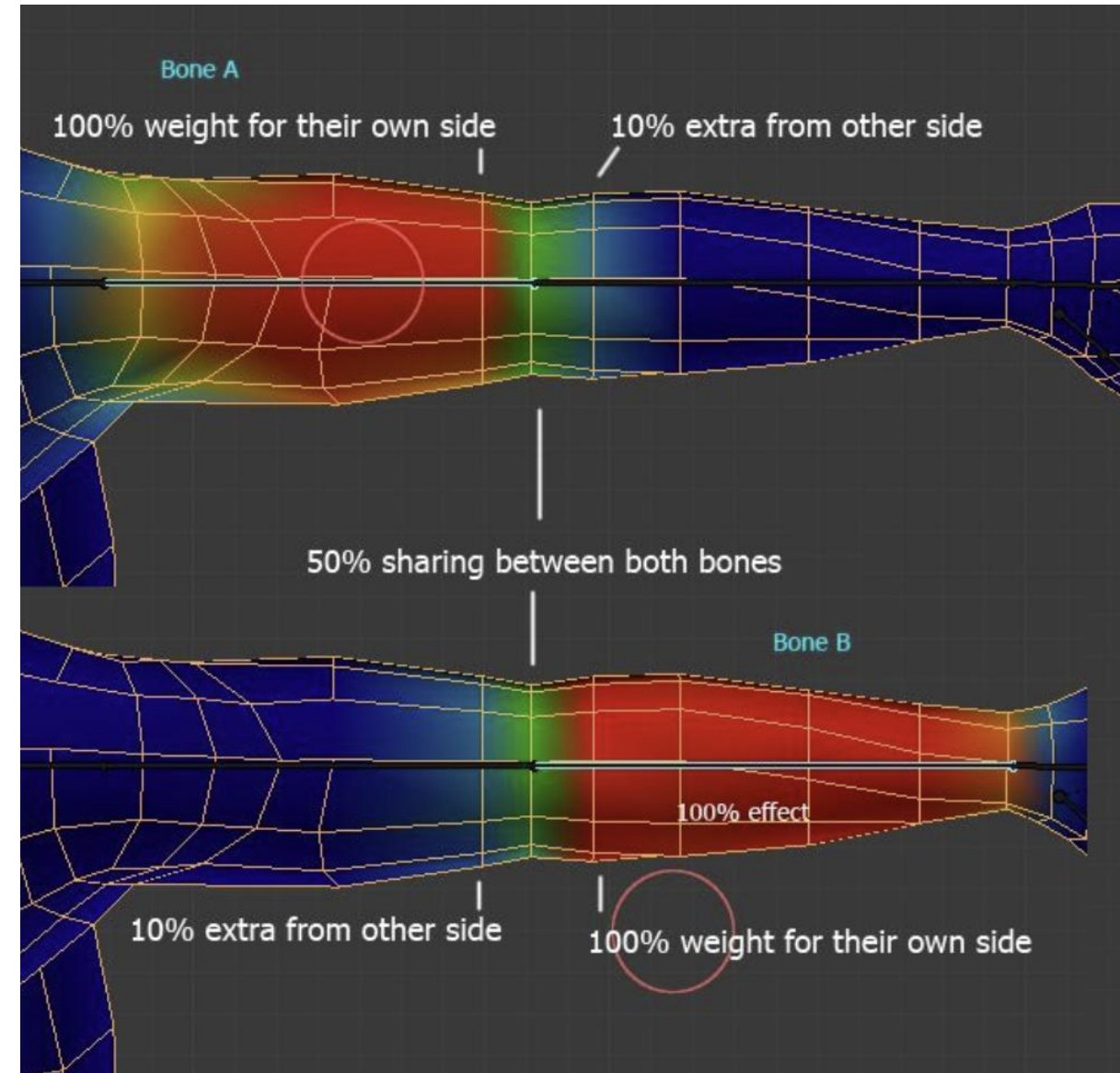
Skinned Mesh

How do we combine the skeleton and surface mesh?

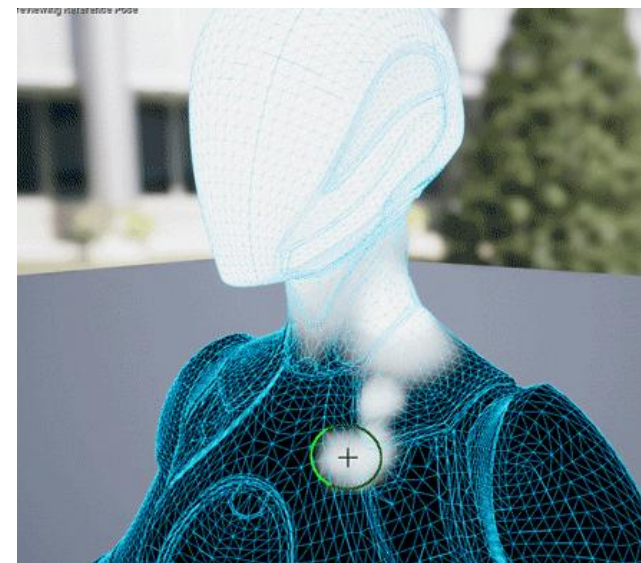
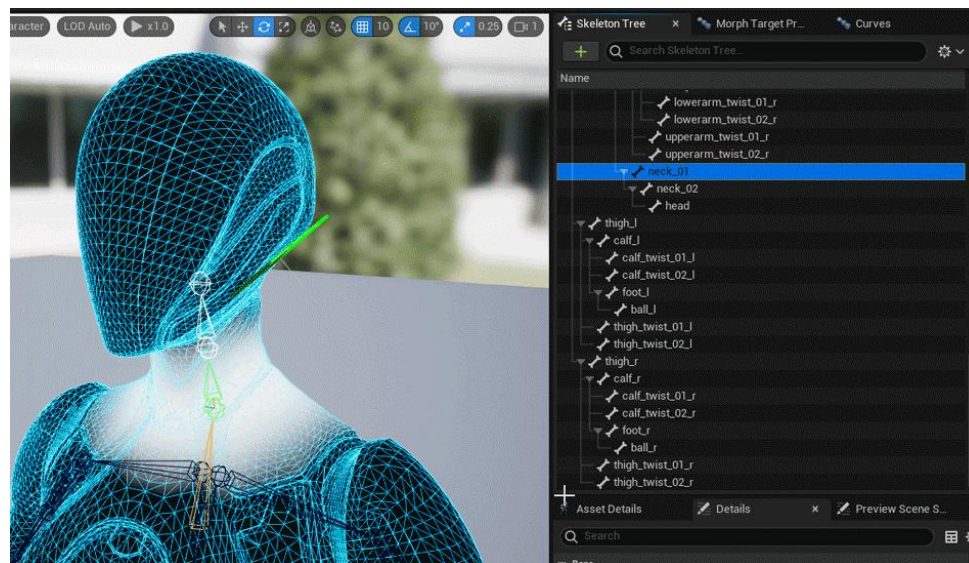
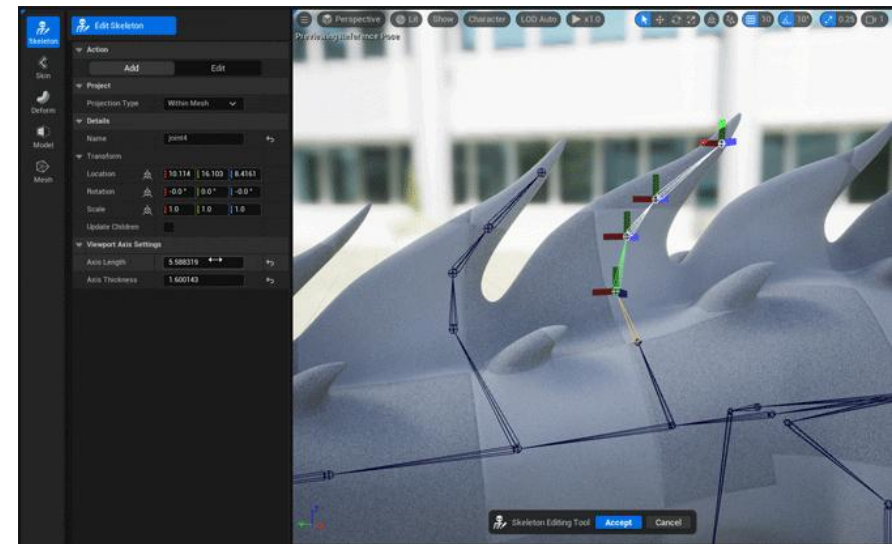
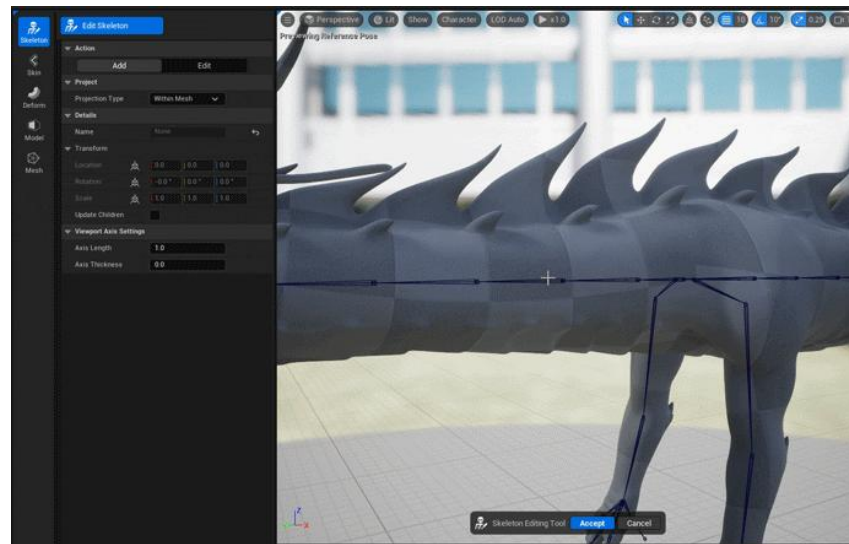
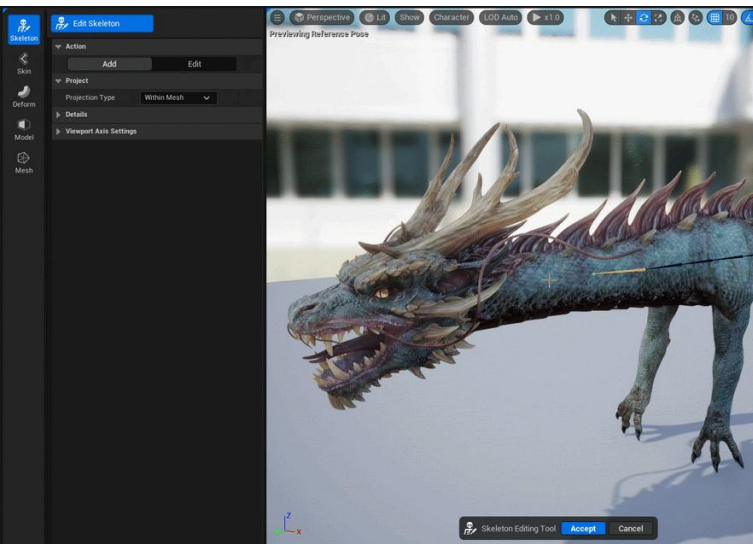


Skinned Mesh

- A mesh that is tightly coupled with the skeleton
 - Also called “enveloped mesh” or “skeletal mesh”
- The vertices of the mesh geometry must be assigned to the **appropriate** bones
 - A group of vertices is mapped to its corresponding bone
 - The ‘connected’ or ‘joint’ vertices may be deformed with respect to
 - The weights of the vertices connected to the bone
 - <https://youtu.be/SCWZr-It1bE?si=v5yuwLktovgOZfIG&t=47>



Unreal Engine Skeletal Editor

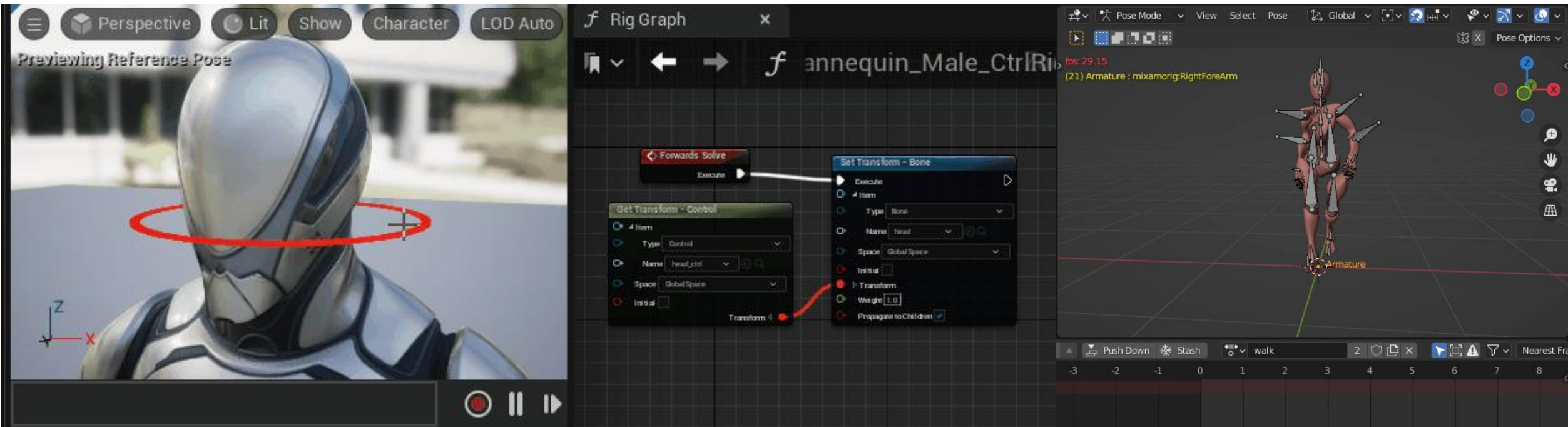


MUST REFER TO *** <https://dev.epicgames.com/community/learning/tutorials/0lpO/unreal-engine-skeletal-editor-usage-guide>

MoCap (Motion Capture) to Rigging <Feat. Retargeting>

What is RIG?

- **Rig** is to configure the skeleton to a unique pose
 - Moving joints may affect other bones belonging to its skeleton
 - Generally composed of both forward kinematics and inverse kinematics parts that may interact with each other
 - Kinematics for animation



Try rigging in some tools such as Blender or Unreal Engine...

Rigging with MoCap

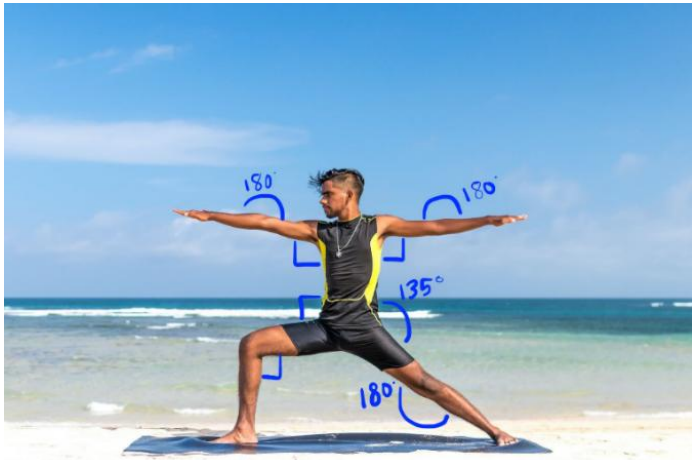
- Classical rigging is performed manually
- Motion Capture (MoCap) can be used for rigging!
 - Captured landmarks are mapped to character's skeleton joints
 - Define a local transform (rotation and translation) of each joint



Estimating Joint (Bone) Angles

- It is noted that the pose landmarks have only position and their link information
 - For animation, we must rig a skeleton using the pose landmarks
 - Mapping the landmarks to the skeletal joints
 - Rigging the joints by using **the local transform of a rotation and a translation**
 - Setting up the startup reference (normally use T pose)
 - **The position (as an offset) of each joint is determined w.r.t. its reference frame (its direct parent frame)**
 - Computing **the rotation** of each joint
 - Determines **the orientation of its bones (connecting the children joints)**

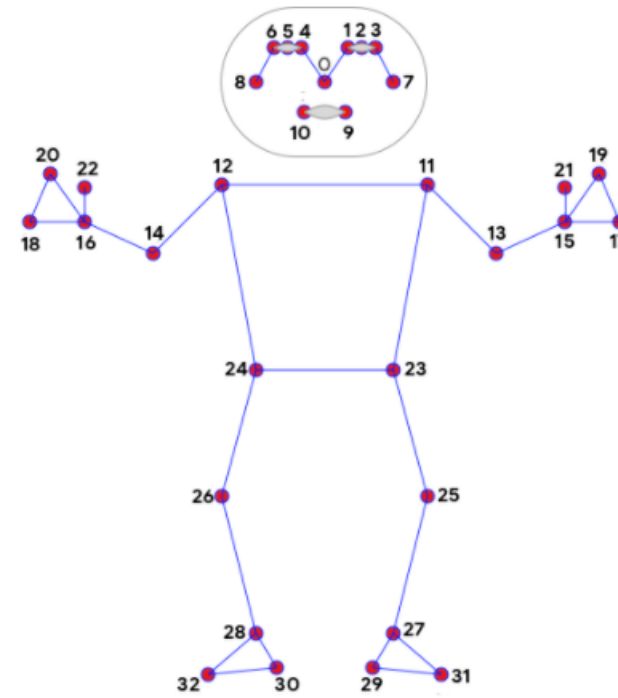
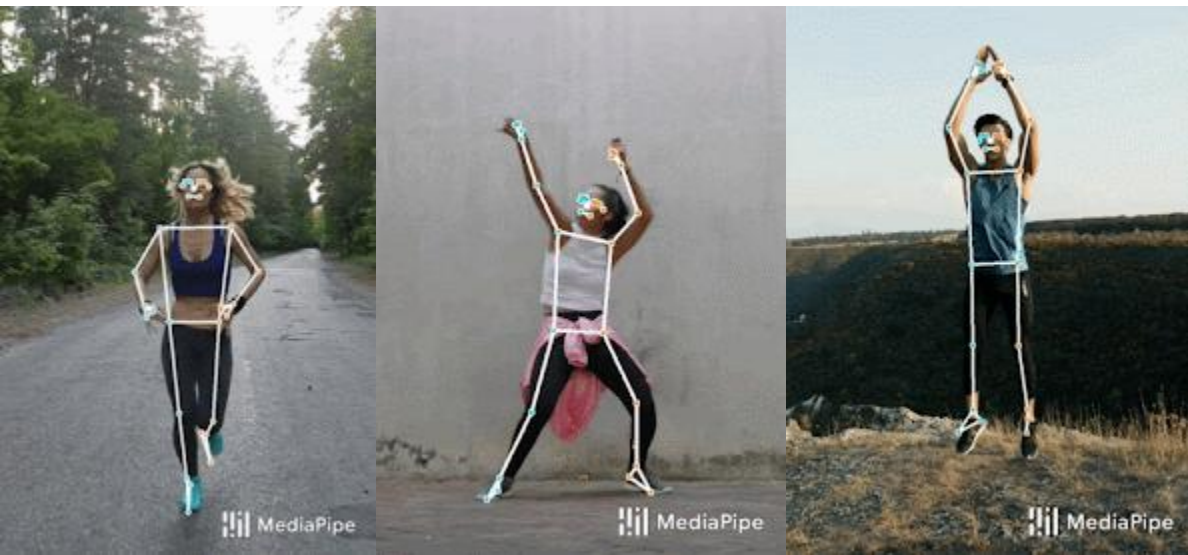
key point to understand the skeleton-based kinematics



Note: the rotation does not describe the figure's in-between angle... then what is it?

Motion Capture and Skeleton

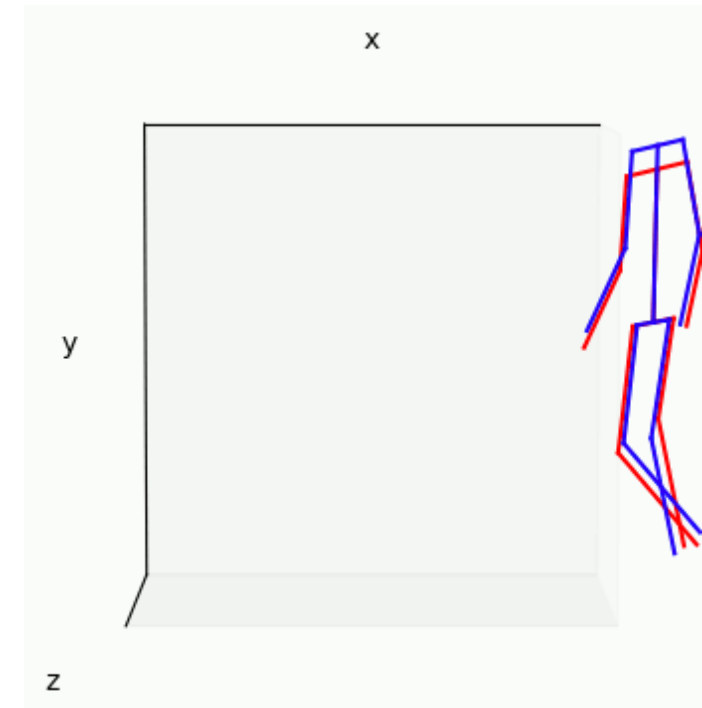
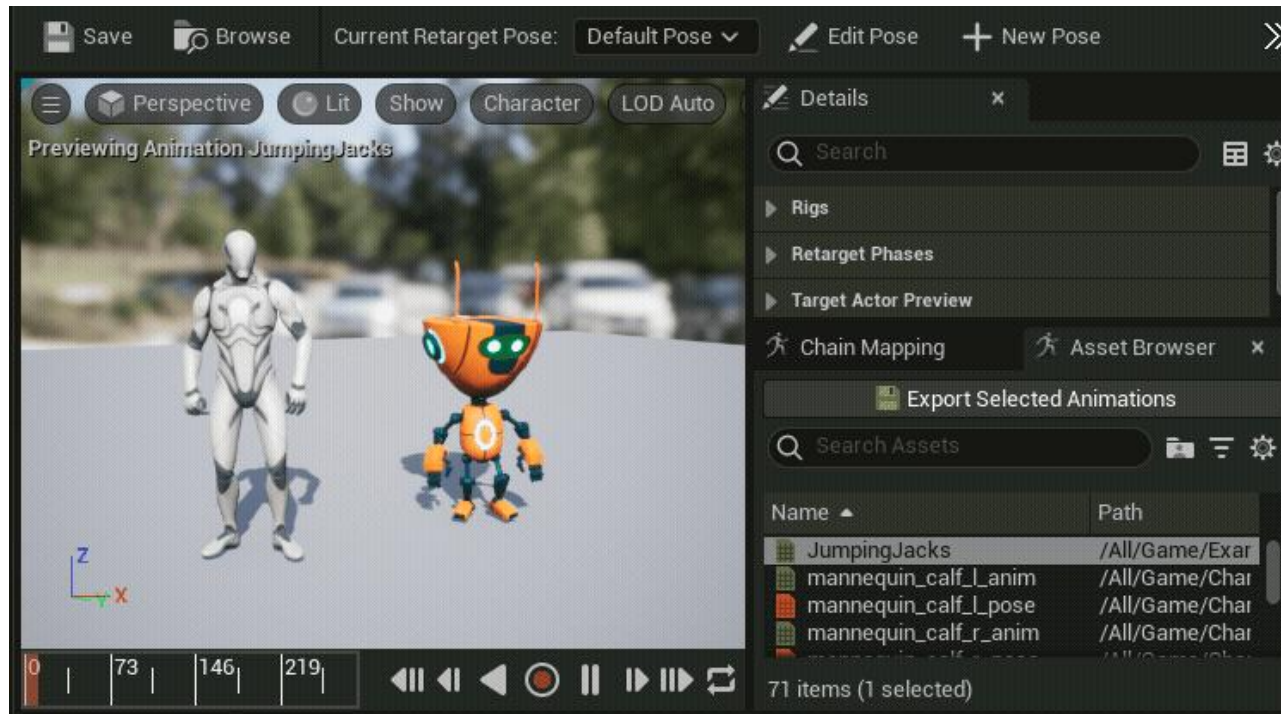
- Pose tracking solutions (by MoCap) provide
 - Landmarks and their connection information (links or lines), not skeleton!
 - Worth reading references
 - <https://ai.googleblog.com/2020/08/on-device-real-time-body-pose-tracking.html>
 - <https://cocodataset.org/#keypoints-2020>
- **Mission: “Rig a skeleton using the pose landmarks”**



- | | |
|--------------------|----------------------|
| 0. nose | 17. left_pinky |
| 1. left_eye_inner | 18. right_pinky |
| 2. left_eye | 19. left_index |
| 3. left_eye_outer | 20. right_index |
| 4. right_eye_inner | 21. left_thumb |
| 5. right_eye | 22. right_thumb |
| 6. right_eye_outer | 23. left_hip |
| 7. left_ear | 24. right_hip |
| 8. right_ear | 25. left_knee |
| 9. mouth_left | 26. right_knee |
| 10. mouth_right | 27. left_ankle |
| 11. left_shoulder | 28. right_ankle |
| 12. right_shoulder | 29. left_heel |
| 13. left_elbow | 30. right_heel |
| 14. right_elbow | 31. left_foot_index |
| 15. left_wrist | 32. right_foot_index |
| 16. right_wrist | |

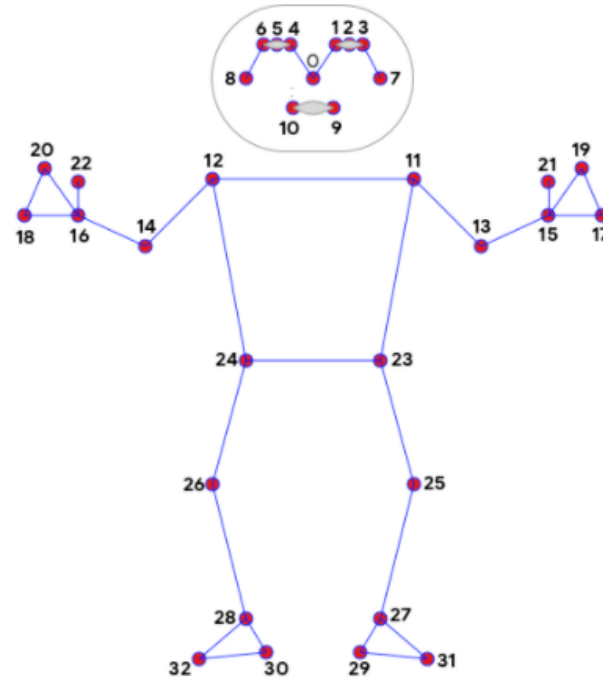
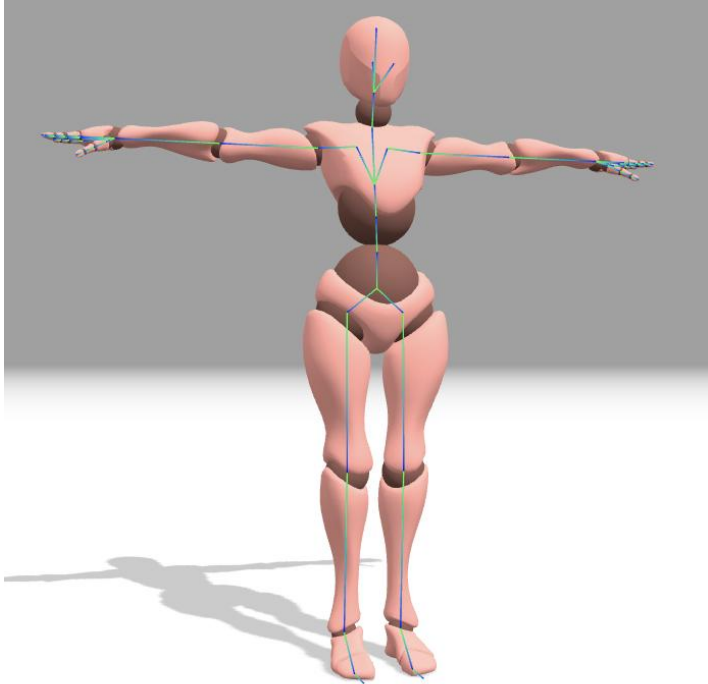
Estimating Joint (Bone) Angles

- This approach is useful for retargeting a pose to a standard skeleton
 - Different subject with different proportions
 - Reduce the noisy and inconsistent pose landmarks
- You may refer to the source at
 - https://temugeb.github.io/python/motion_capture/2021/09/16/joint_rotations.html

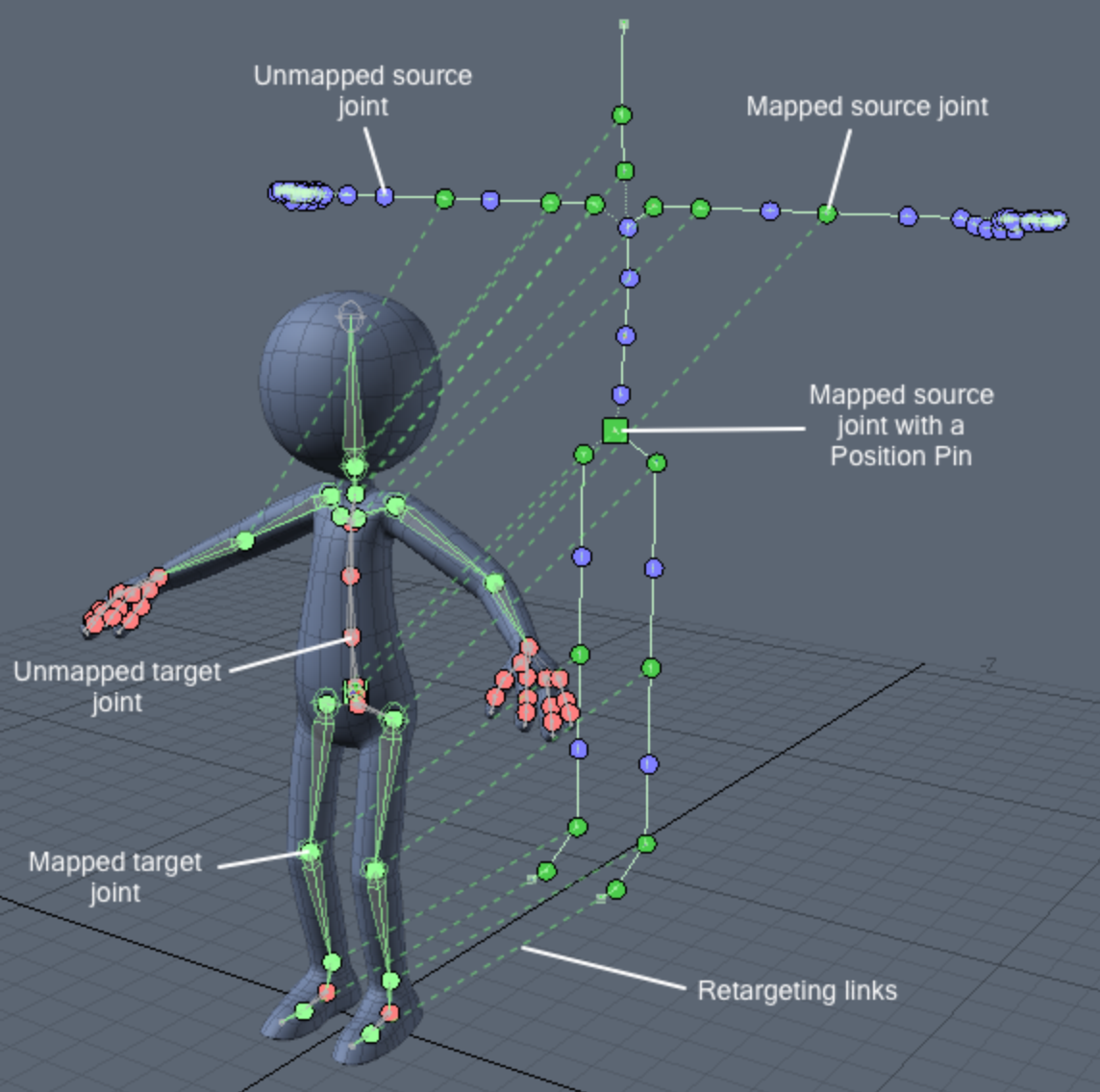


MoCap Landmarks to Skeletal Animation

- Now, we can rig the character's skeleton using the MoCap landmarks
- For more stable rigging, we can use
 - New bones by interpolation
 - IK solver for the natural animation (partial rigs leading to kinematic movements)
 - Physics



- | | |
|--------------------|----------------------|
| 0. nose | 17. left_pinky |
| 1. left_eye_inner | 18. right_pinky |
| 2. left_eye | 19. left_index |
| 3. left_eye_outer | 20. right_index |
| 4. right_eye_inner | 21. left_thumb |
| 5. right_eye | 22. right_thumb |
| 6. right_eye_outer | 23. left_hip |
| 7. left_ear | 24. right_hip |
| 8. right_ear | 25. left_knee |
| 9. mouth_left | 26. right_knee |
| 10. mouth_right | 27. left_ankle |
| 11. left_shoulder | 28. right_ankle |
| 12. right_shoulder | 29. left_heel |
| 13. left_elbow | 30. right_heel |
| 14. right_elbow | 31. left_foot_index |
| 15. left_wrist | 32. right_foot_index |
| 16. right_wrist | |



A **Character** has a shape that can be mapped to a specific **Skeleton**



Skeleton's Animation can be applied to the **Character**

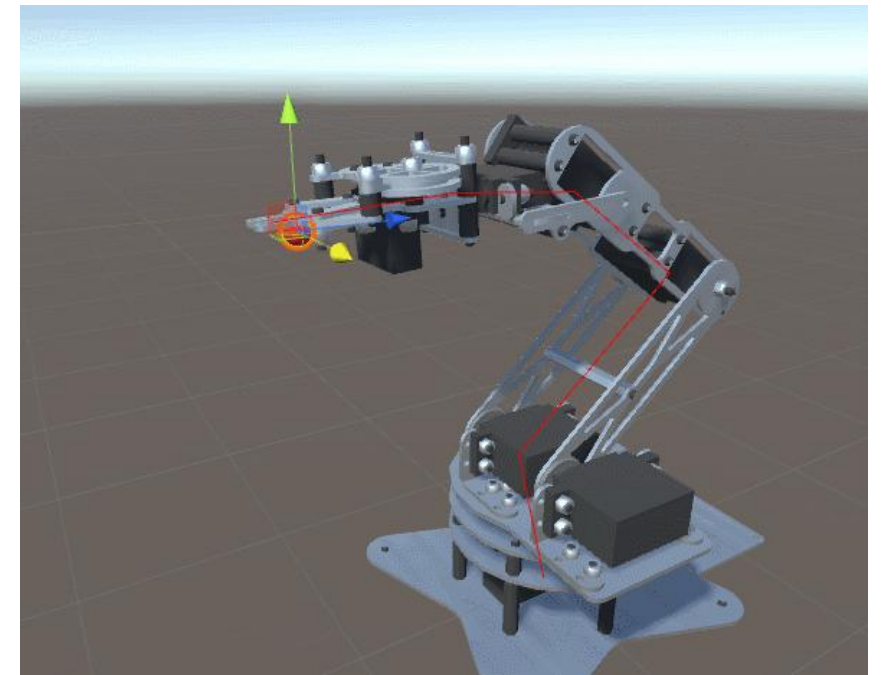
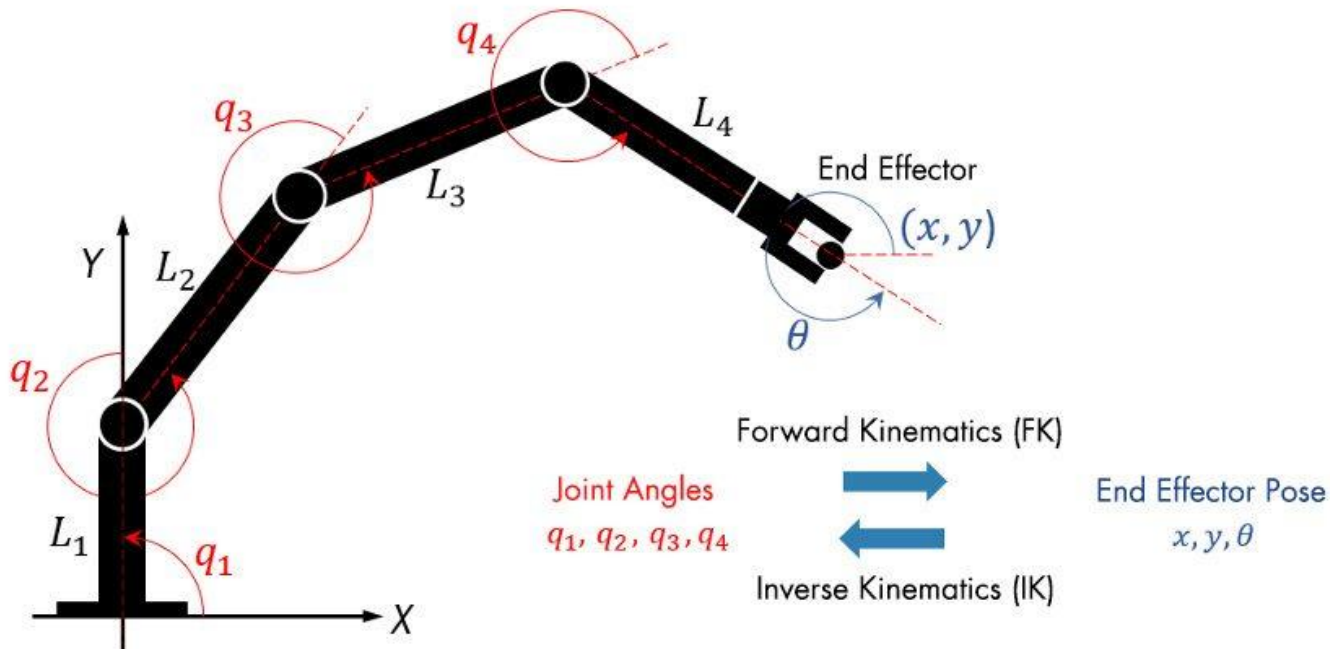


- <https://learn.foundry.com/modo/content/help/pages/animation/tools/retargeting.html>
- <https://medium.com/@nvidiaomniverse/what-is-animation-retargeting-4aadab383032>

Animation with Skeleton - Kinematics

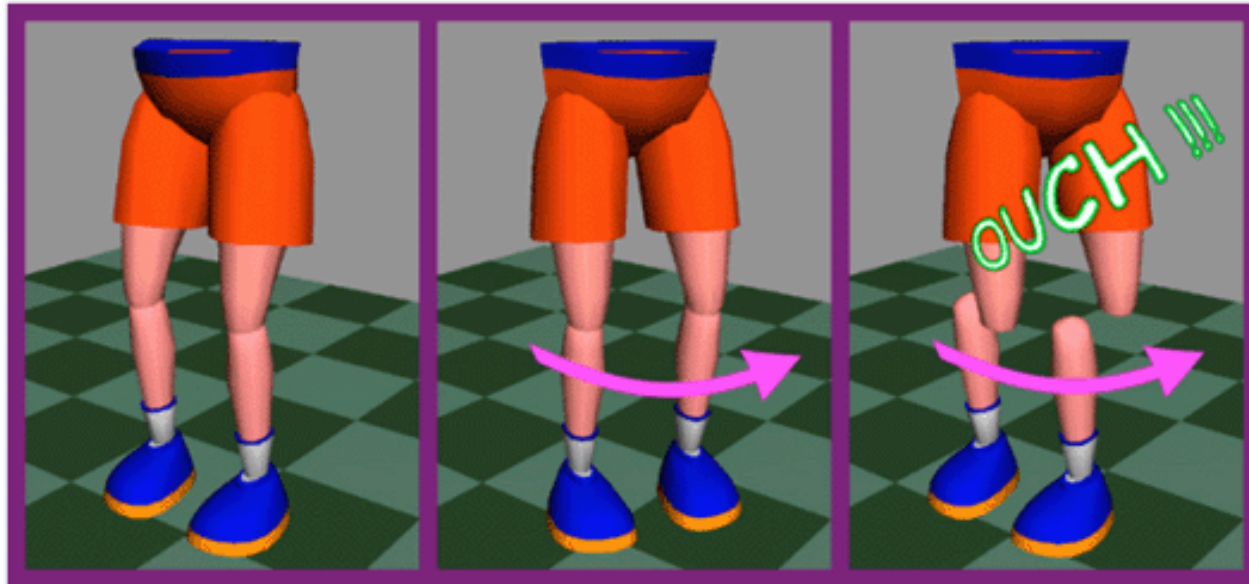
Kinematics

- Classical mechanics that describes the motion of points, bodies (objects), and systems of bodies (groups of objects) without considering the forces that cause them to move
 - Referred to as Geometry of Motion
 - Forward Kinematics (FK) vs. **Inverse Kinematics (IK)**
- Kinematics for Physical skeleton:
 - Transform a **rigid structure** with tendons at each joint to hold it all together



Forward Kinematics

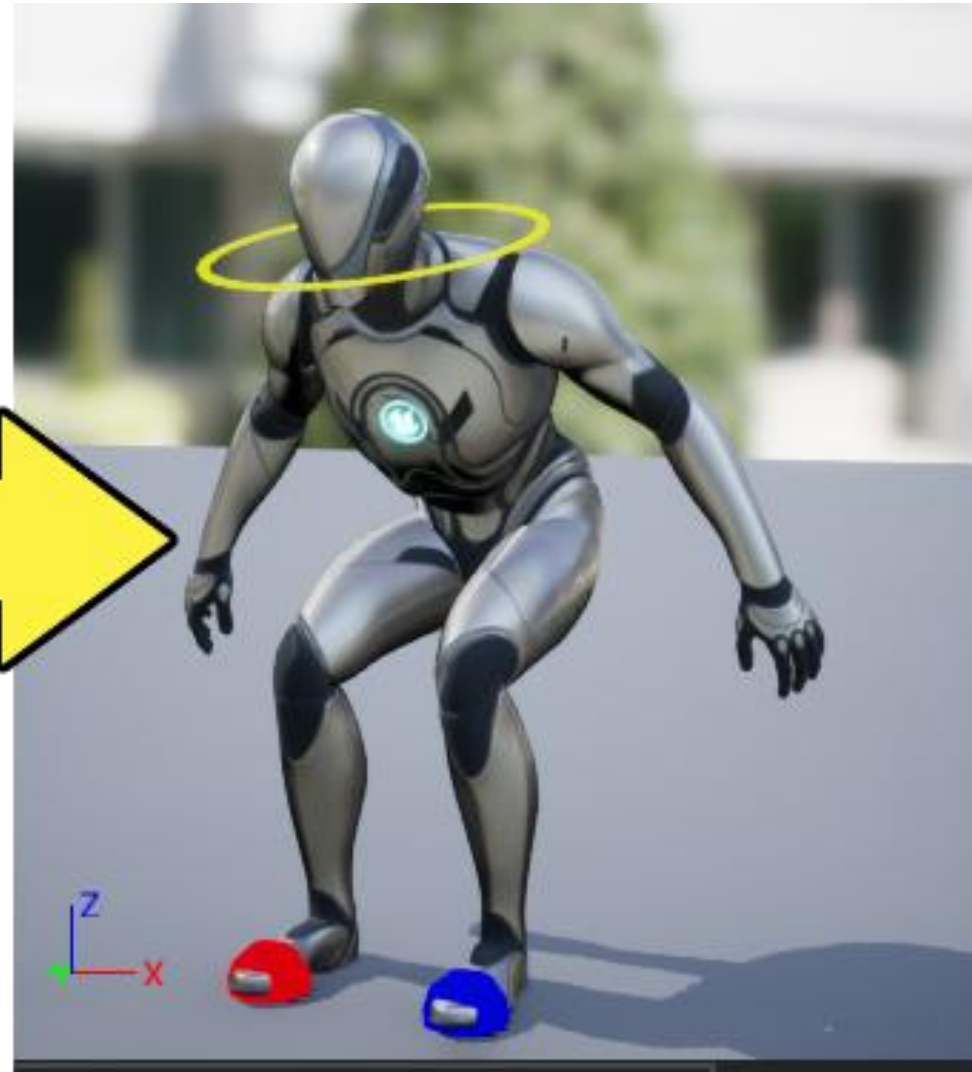
- FK is great for basic animation but very tedious for configuring some poses
 - Top-down manipulation (manual)
 - Without consideration of complex skeleton constraints, it will show aesthetically unpleasing results



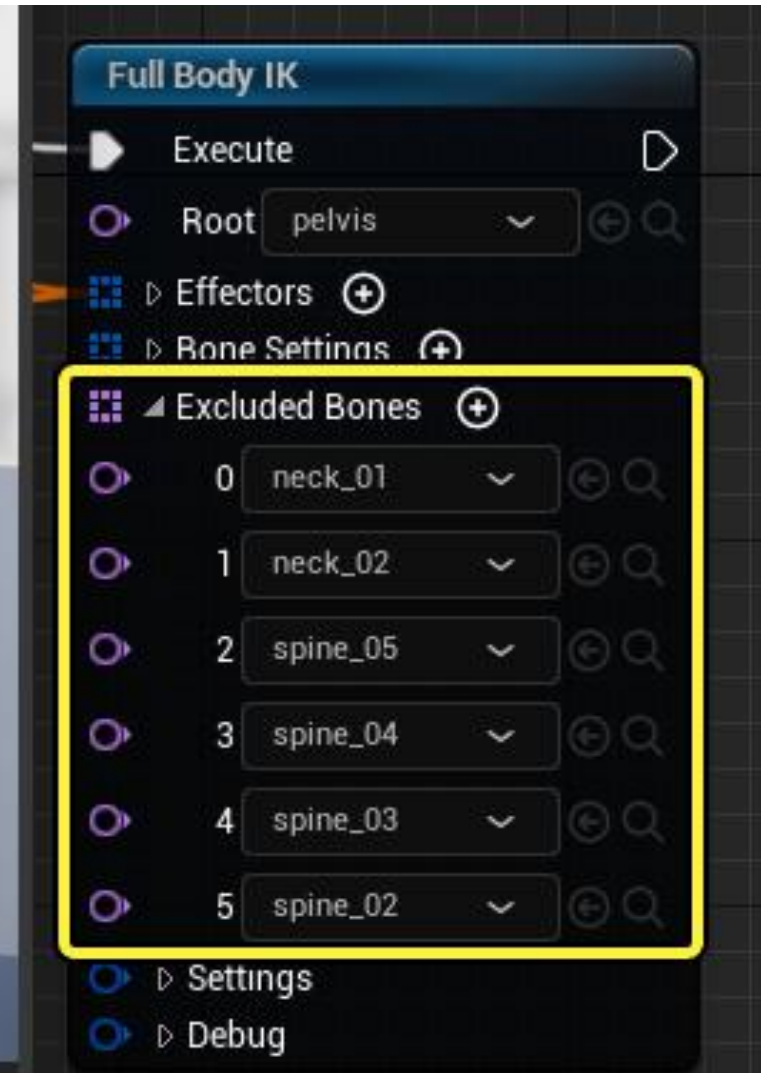
Forward Kinematics vs. Inverse Kinematics



FK

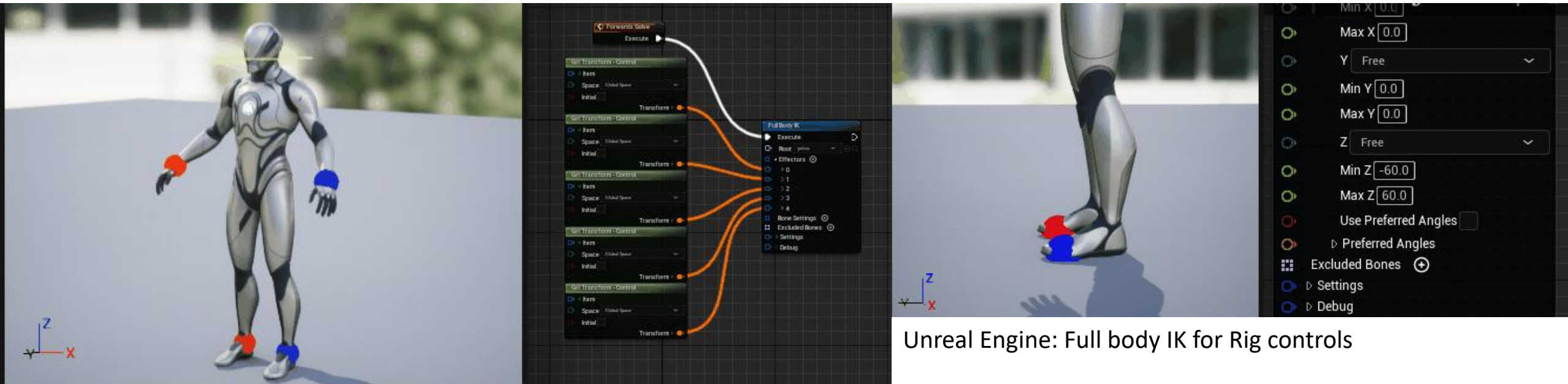


Unreal Engine: Full body IK for Rig controls



Inverse Kinematics

- CG Skeleton allows to transfer control from the top of the hierarchy to the bottom
 - Bottom-up 'process' (computation) - **Inverse Kinematics (IK)**
 - Finding an optimal (shortest) transforms to reach the final pose 😊 (Solver)
 - Two links (easy and straightforward) vs. more than two links (challenging)
- Inverse Kinematics (IK) compared to Forward Kinematics (FK)
 - Much more intuitive and much less time-consuming process than dealing with top-down rotations



Unreal Engine: Full body IK for Rig controls

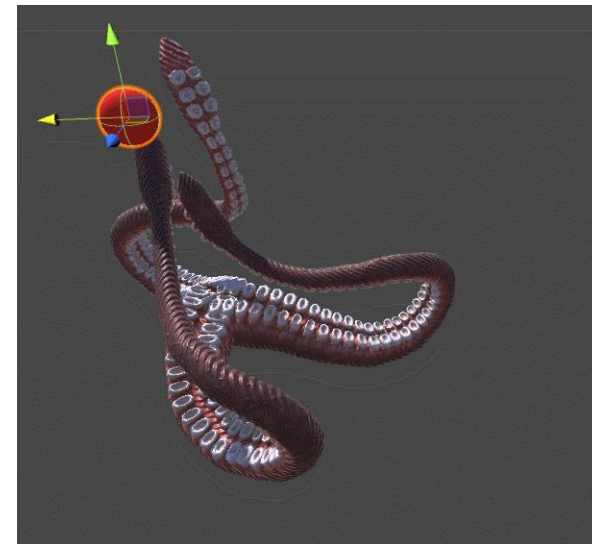
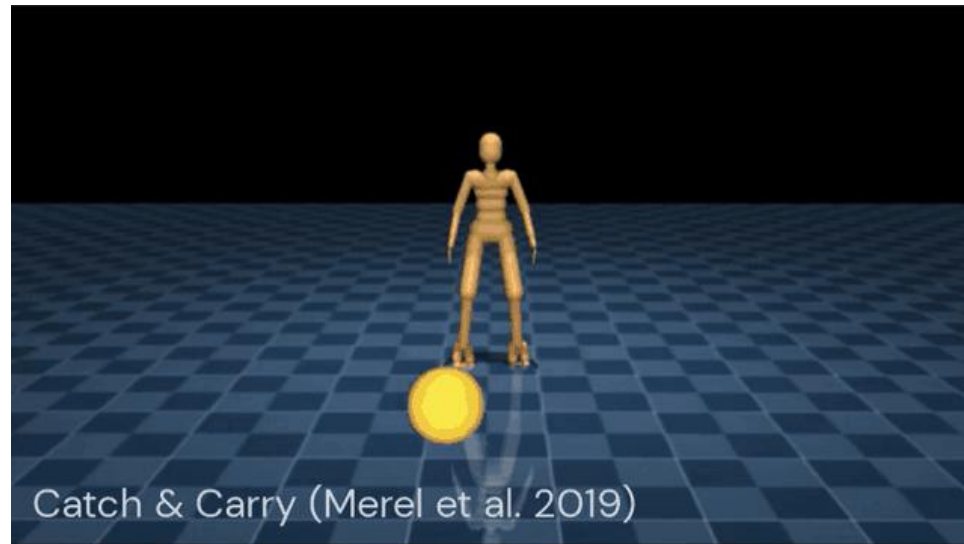
Advantages of Inverse Kinematics

- A simple generation of a pose (e.g., simply place the hand, leading to the rest of arm follows the hand)
- A natural control structure for deforming the surface
 - The control points (group of vertices mapped to bones or joints) deform the geometry
- An easy employment of constraints (e.g., magnetic or gravity forces combined with mathematical formulas)

**Active
Ragdoll**



**Normal
Ragdoll**



Animation Retargeting Practice (feat. AI)

Animation Retargeting Process

- Convert **Static mesh** to **Skeletal mesh** (Skinned Mesh)
- Add **skeleton** and bind it to skeletal mesh
 - Using **Skeleton Editor**
 - Binding can be accomplished automatically (using distance metric)
- Assign **skin weights** (for natural deformation of the skeleton mesh)
- Retargeting (source to target)
 - **IK retargeting** : mapping source's **IK rig** to target IK rig
 - IK rig can be simply handled by **bone chains**
 - **IK retargeter** : mapping source's bone chains to target's bone chains
 - Apply animation